



ITSRLL
INSTITUTO TECNOLÓGICO SUPERIOR
DE LA REGIÓN DE LOS LLANOS

Ingeniería Mecatrónica

PROGRAMACIÓN AVANZADA

Enero – Junio 2025
M.C. Osbaldo Aragón Banderas

UNIDAD: 1

Actividad número: A4

Nombre de actividad:

Reporte de Programa en Jupyter Notebook – Evaluación de
Métodos de Ordenamiento

Actividad realizada por:

Roberto Jair Arteaga Valenzuela

Guadalupe Victoria, Durango

Fecha de entrega: 12 de febrero de 2025

Implementación y Evaluación de Algoritmos de Ordenamiento en Python

Introducción

El ordenamiento de datos es una operación fundamental en la computación, con aplicaciones en diversas áreas, incluida la robótica. En esta actividad, se implementan y analizan dos algoritmos de ordenamiento: Burbuja y Quicksort, utilizando Python en un entorno Jupyter Notebook. Además, se estudia su rendimiento en conjuntos de datos de diferentes tamaños. Este análisis permite comprender la eficiencia de los algoritmos y su impacto en la optimización del software. Además, se fomenta el uso de repositorios en GitHub como herramienta profesional para documentar y compartir el trabajo desarrollado.

Objetivo

Utilizar Jupyter Notebook para implementar y evaluar dos métodos de ordenamiento (Burbuja y Quicksort) en Python, analizando su rendimiento. Esta actividad refuerza el conocimiento de algoritmos, fundamentales para la optimización del software en aplicaciones de robótica. Además, se busca que los estudiantes practiquen la creación y organización de repositorios en GitHub, con el fin de que sirvan como apoyo en su currículo profesional.

Desarrollo

Implementación de Algoritmos de Ordenamiento

A continuación, se presentan las implementaciones en Python de los algoritmos de ordenamiento Burbuja y Quicksort:

```
import timeit
import random

# Implementación del algoritmo de ordenamiento Burbuja
def metodo_burbuja(lista):
    for i in range(len(lista)):
        for j in range(0, len(lista) - i - 1):
            if lista[j] > lista[j + 1]:
                lista[j], lista[j + 1] = lista[j + 1], lista[j]
    return lista

# Implementación del algoritmo de ordenamiento Quicksort
def quicksort(lista):
    if len(lista) <= 1:
        return lista
    else:
        pivote = lista[len(lista) // 2]
        izquierda = [x for x in lista if x < pivote]
        centro = [x for x in lista if x == pivote]
        derecha = [x for x in lista if x > pivote]
        return quicksort(izquierda) + centro + quicksort(derecha)

# Generar listas de diferentes tamaños
listas = {
    100: [random.randint(0, 10000) for _ in range(100)],
    1000: [random.randint(0, 10000) for _ in range(1000)],
    5000: [random.randint(0, 10000) for _ in range(5000)],
    7500: [random.randint(0, 10000) for _ in range(7500)],
    10000: [random.randint(0, 10000) for _ in range(10000)]
}

# Pruebas de rendimiento
print("Comparación de rendimiento con diferentes tamaños de listas:")
for tamaño, lista in listas.items():
    tiempo_burbuja = timeit.timeit(lambda: metodo_burbuja(lista.copy()), number=1)
    tiempo_quicksort = timeit.timeit(lambda: quicksort(lista.copy()), number=1)
    print(f"Tamaño {tamaño}: Burbuja: {tiempo_burbuja:.5f} s | Quicksort: {tiempo_quicksort:.5f} s")
```

Procedimiento

- **Generación de listas aleatorias:** Se crean listas de diferentes tamaños con valores generados aleatoriamente, 100, 100, 500, 7500 y 10000.
- **Ordenamiento con Burbuja:** Se implementa el algoritmo Burbuja, que recorre repetidamente la lista y compara elementos adyacentes, intercambiándolos si están en el orden incorrecto. Este proceso se repite hasta que la lista esté ordenada.
- **Ordenamiento con Quicksort:** Se implementa Quicksort, que elige un pivote y divide la lista en tres partes: elementos menores, iguales y mayores al pivote. Luego, aplica recursivamente el mismo procedimiento a las sublistas izquierda y derecha.
- **Medición del tiempo de ejecución:** Se utiliza `timeit` para calcular el tiempo de ejecución de cada algoritmo en diferentes tamaños de listas.
- **Comparación de resultados:** Se analizan los tiempos de ejecución obtenidos para determinar la eficiencia de cada método.

Pruebas de Rendimiento

Se generaron listas de diferentes tamaños con valores aleatorios y se realizaron pruebas de rendimiento utilizando el módulo `timeit`. Los tiempos de ejecución obtenidos permiten comparar la eficiencia de ambos algoritmos.

```
# Generar Listas de diferentes tamaños
listas = {
    100: [random.randint(0, 10000) for _ in range(100)],
    1000: [random.randint(0, 10000) for _ in range(1000)],
    5000: [random.randint(0, 10000) for _ in range(5000)],
    7500: [random.randint(0, 10000) for _ in range(7500)],
    10000: [random.randint(0, 10000) for _ in range(10000)]
}

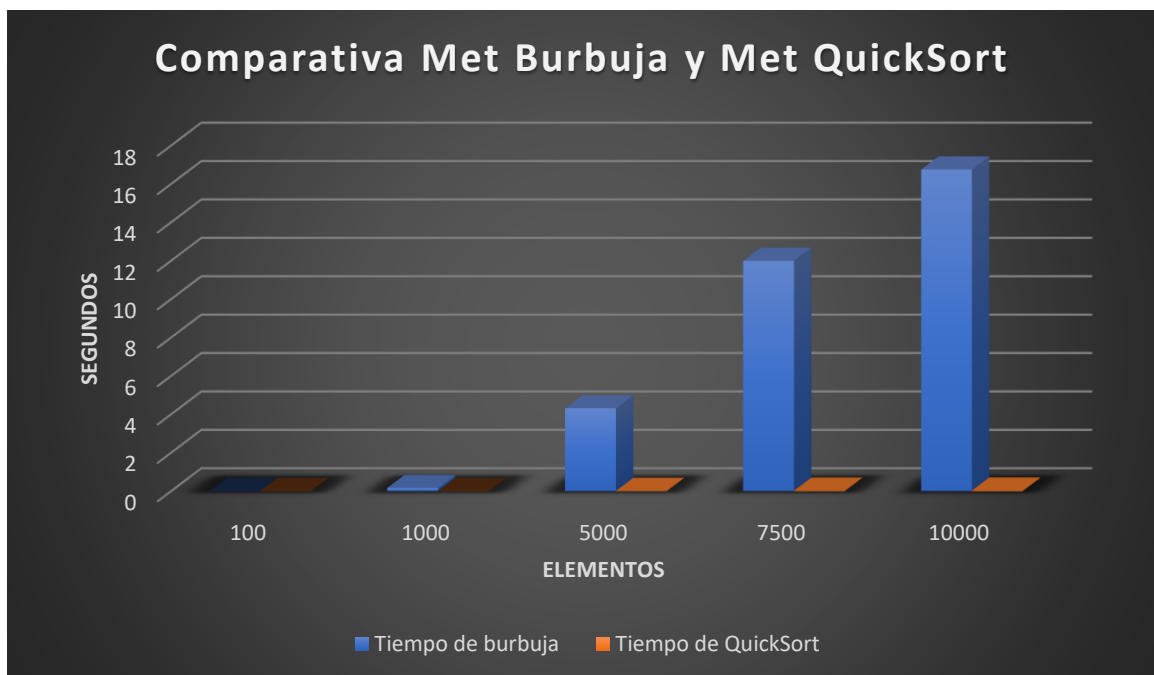
# Pruebas de rendimiento
print("Comparación de rendimiento con diferentes tamaños de listas:")
for tamaño, lista in listas.items():
    tiempo_burbuja = timeit.timeit(lambda: metodo_burbuja(lista.copy()), number=1)
    tiempo_quicksort = timeit.timeit(lambda: quicksort(lista.copy()), number=1)
    print(f"Tamaño {tamaño}: Burbuja: {tiempo_burbuja:.5f} s | Quicksort: {tiempo_quicksort:.5f} s")
```

Resultados y Análisis

Los resultados obtenidos muestran diferencias significativas en el tiempo de ejecución entre ambos algoritmos. El algoritmo de Burbuja, debido a su complejidad cuadrática $O(n^2)$, resulta ineficiente para listas grandes, mientras que Quicksort, con una complejidad $O(n \log n)$ en promedio, muestra una mejor eficiencia y escalabilidad.

A continuación, se resumen los tiempos de ejecución:

Tamaño de lista	Tiempo de burbuja en s	Tiempo de QuickSort en s
100	0.00139	0.00025
1000	0.16961	0.00360
5000	4.32197	0.01777
7500	12.00376	0.02709
10000	16.75661	0.03674



```
Comparación de rendimiento con diferentes tamaños de listas:  
Tamaño 100: Burbuja: 0.00139 s | Quicksort: 0.00025 s  
Tamaño 1000: Burbuja: 0.16961 s | Quicksort: 0.00360 s  
Tamaño 5000: Burbuja: 4.32197 s | Quicksort: 0.01777 s  
Tamaño 7500: Burbuja: 12.00376 s | Quicksort: 0.02709 s  
Tamaño 10000: Burbuja: 16.75661 s | Quicksort: 0.03674 s
```

Figura 1 Resultados del análisis

A partir de estos resultados, se observa lo siguiente:

Crecimiento del tiempo de ejecución:

- El método Burbuja muestra un aumento exponencial en el tiempo de ejecución conforme crece el tamaño de la lista.
- Quicksort, en cambio, mantiene un crecimiento mucho más controlado y estable, incluso con listas más grandes.

Complejidad Temporal:

- Burbuja: $O(n^2)$ → A medida que el tamaño de la lista se incrementa, el número de comparaciones y swaps crece de forma cuadrática, volviéndolo inviable para listas grandes.
- Quicksort: $O(n \log n)$ en promedio → La estrategia de dividir y conquistar permite que el tiempo de ejecución crezca de forma logarítmica, lo que lo hace mucho más eficiente.

Diferencia en tiempos:

- Con 100 elementos, la diferencia entre ambos métodos es pequeña, pero aún así Quicksort es aproximadamente 5.5 veces más rápido que Burbuja.
- Con 1000 elementos, Quicksort es aproximadamente 47 veces más rápido que Burbuja.
- Con 5000 elementos, la diferencia es aún más notoria: Quicksort es 243 veces más rápido.
- Con 10,000 elementos, el método Burbuja tarda más de 16 segundos, mientras que Quicksort solo 0.03674 segundos, lo que implica que Quicksort es alrededor de 456 veces más rápido en este caso.

Los resultados muestran claramente que Quicksort es significativamente más eficiente que el método Burbuja, especialmente cuando el tamaño de la lista aumenta. Esto confirma que los algoritmos con complejidad $O(n \log n)$ son preferibles a aquellos con $O(n^2)$ para manejar grandes volúmenes de datos.

Conclusión

Este experimento reafirma la importancia de elegir algoritmos eficientes para el procesamiento de datos. Los resultados demuestran que Quicksort es significativamente más rápido que el método Burbuja, especialmente a medida que el tamaño de los datos aumenta. Esto se debe a que la complejidad de Burbuja es $O(n^2)$, lo que provoca que su tiempo de ejecución crezca exponencialmente conforme aumenta el número de elementos. En contraste, Quicksort tiene una complejidad promedio de $O(n \log n)$, lo que le permite manejar grandes volúmenes de datos de manera mucho más eficiente.

El uso de algoritmos eficientes como Quicksort es crucial en aplicaciones donde el tiempo de procesamiento es un factor determinante, como en sistemas robóticos, análisis de datos y optimización de software. Además, la documentación del código en Jupyter Notebook y su gestión en GitHub proporcionan un enfoque profesional para el desarrollo de proyectos, fomentando buenas prácticas en la programación y colaboración en equipo.

Link de github

<https://github.com/Jair-Artreaga/Implementacion-Evaluacion-Algoritmos-Ordenamiento-Python.git>